Planejamento Técnico Inicial – VITRINEIQ

Primeira entrega (13/11/2026)

Versão 1.1

Desafio 06 · Inteligência comercial para lojistas · Capim Dourado Shopping
1º HackaRun UniCatólica 2026 · Time Nido King



Índice

Resumo da solução
1. Proposta e estrutura inicial das telas
2. Documentação inicial do projeto
2.1 Problema, público e proposta de valor
2.2 Artefatos produzidos
2.3 Fontes de dados
3. Linguagem de programação escolhida
4. Arquitetura proposta
5. Plataforma e tecnologias adotadas
6. Prática sorteada: User Stories
7. Evolução prevista (roadmap)
8. Equipe e uso de IA

Resumo da solução

O VitrineIQ é uma plataforma web que mostra ao lojista do Capim Dourado Shopping quem passa pela loja, por que o cliente entra e sai sem comprar e que fatores (horário, clima, eventos do shopping) influenciam suas vendas. A partir desses sinais, um Agente de IA gera um plano de ação em linguagem simples. Nenhum dado pessoal é coletado: o lojista vê apenas números agregados.

Item da entrega	Decisão
Telas	7 telas: Painel do Lojista, Gêmeo Digital 3D, Sensor de Vitrine, Modo Balcão, Concierge QR, User Stories e LGPD/Auditoria
Linguagem	TypeScript com React no frontend e Python no backend
Arquitetura	3 camadas: frontend React capta sinais anônimos → API Python (FastAPI) calcula o score e consulta o Agente de IA → provedor de IA externo recebe só números agregados
Plataforma	Aplicação web responsiva publicada na Vercel, API Python com FastAPI e Uvicorn, Agente de IA (Gemini no MVP, provedor intercambiável) com fallback heurístico
Prática sorteada	User Stories guiam backlog, Kanban, commits, testes e a própria interface

1. Proposta e estrutura inicial das telas

A interface é uma aplicação web única, com navegação lateral entre sete telas. Ela funciona no notebook da gerência, no tablet do balcão e no celular do visitante. As capturas abaixo são do protótipo já publicado.

Tela	O que mostra	História
Painel do Lojista	Score de Oportunidade (0 a 100), funil da porta (passaram → pararam → entraram → compraram), gráfico de horário de pico de 10h às 22h, simulação de clima e eventos e Agente de IA com campanha relâmpago.	US01
Shopping 3D	Maquete navegável dos dois pavimentos baseada na Planta Mix oficial, com calor de demanda por ala, troca de pavimento, modo de caminhada e alternância para a planta 2D.	US05
Sensor de Vitrine	Conecta uma webcam (ou simulador) e conta, por zona, quem passa no corredor, para na vitrine, entra e sai, sem gravar imagens.	US02
Modo Balcão	O vendedor registra em um toque o motivo da venda perdida: preço, falta de tamanho ou estoque, só olhando, fila.	US03
Concierge QR	Página aberta pelo QR da vitrine, sem login nem CPF: o visitante informa o que procura e o orçamento e recebe o cupom ativo.	US04
User Stories	Mapa das histórias de usuário, com botão que abre a tela que implementa cada uma.	Prática sorteada
LGPD e Auditoria	Regras de privacidade aplicadas e trilha de auditoria com hash dos registros.	RNF02, RNF03

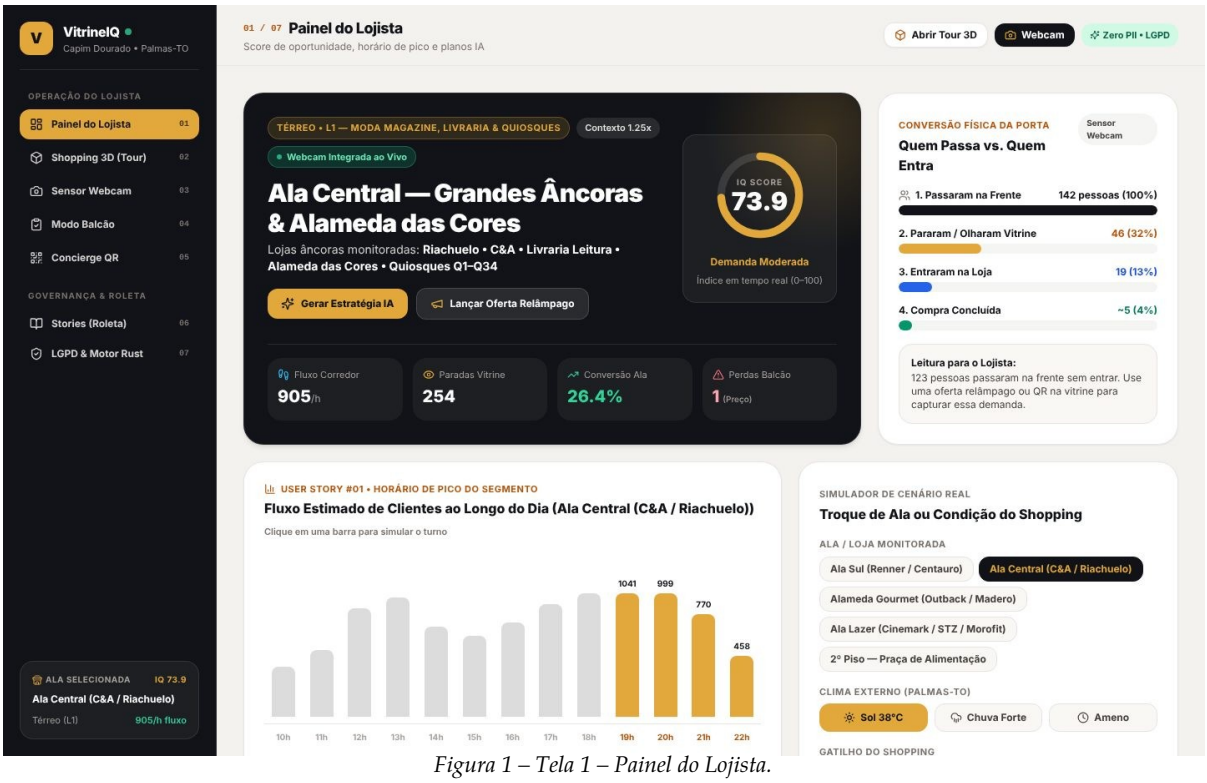


Figura 1 – Tela 1 – Painel do Lojista.

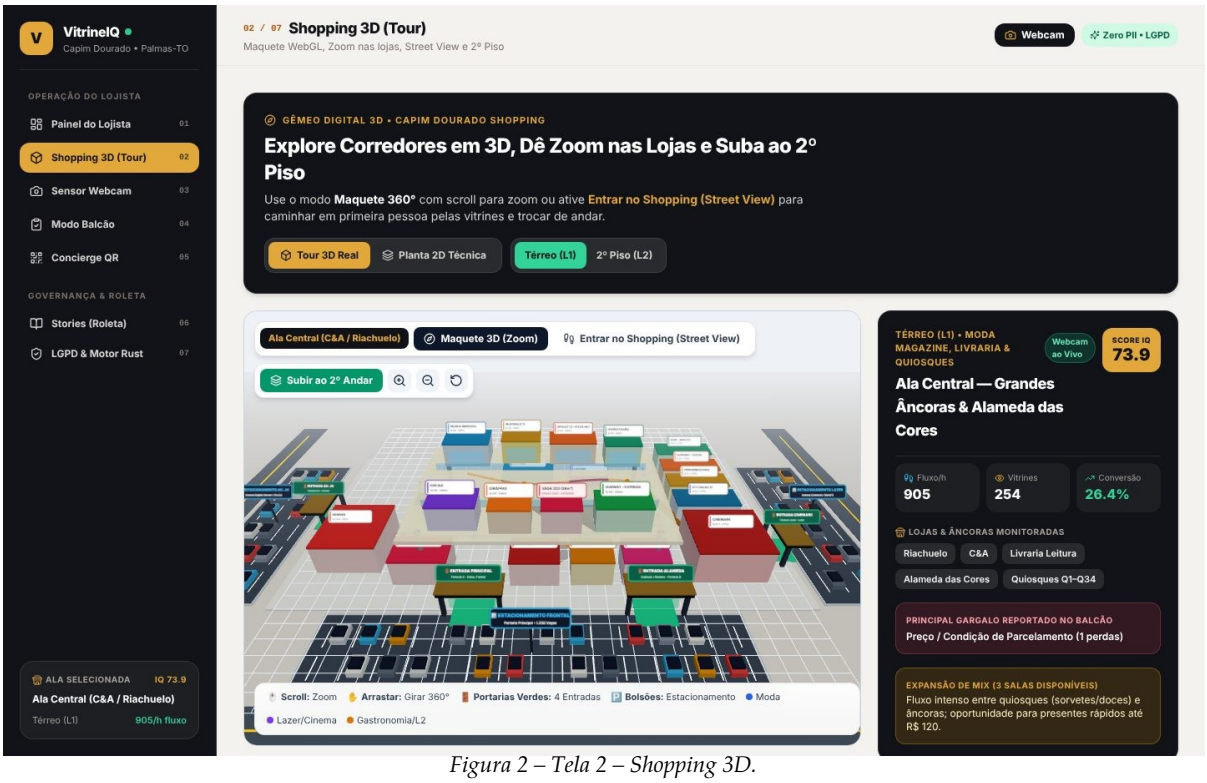


Figura 2 – Tela 2 – Shopping 3D.

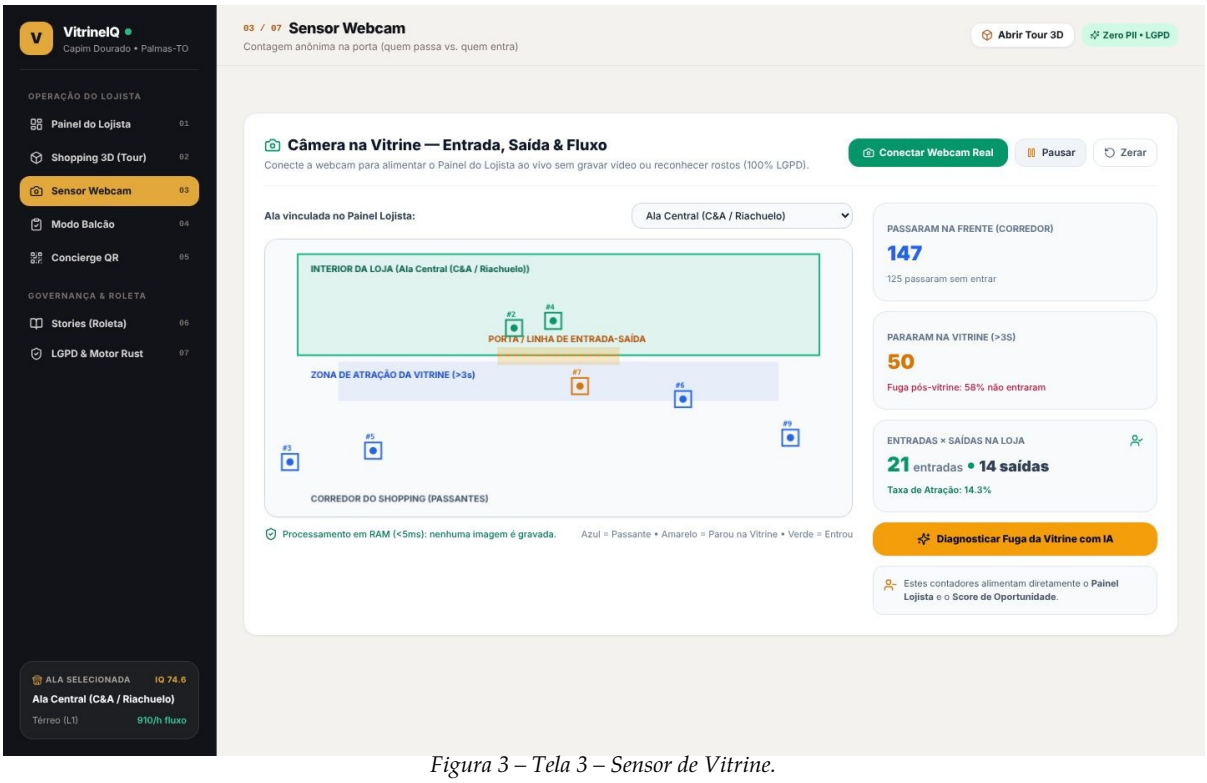


Figura 3 – Tela 3 – Sensor de Vitrine.

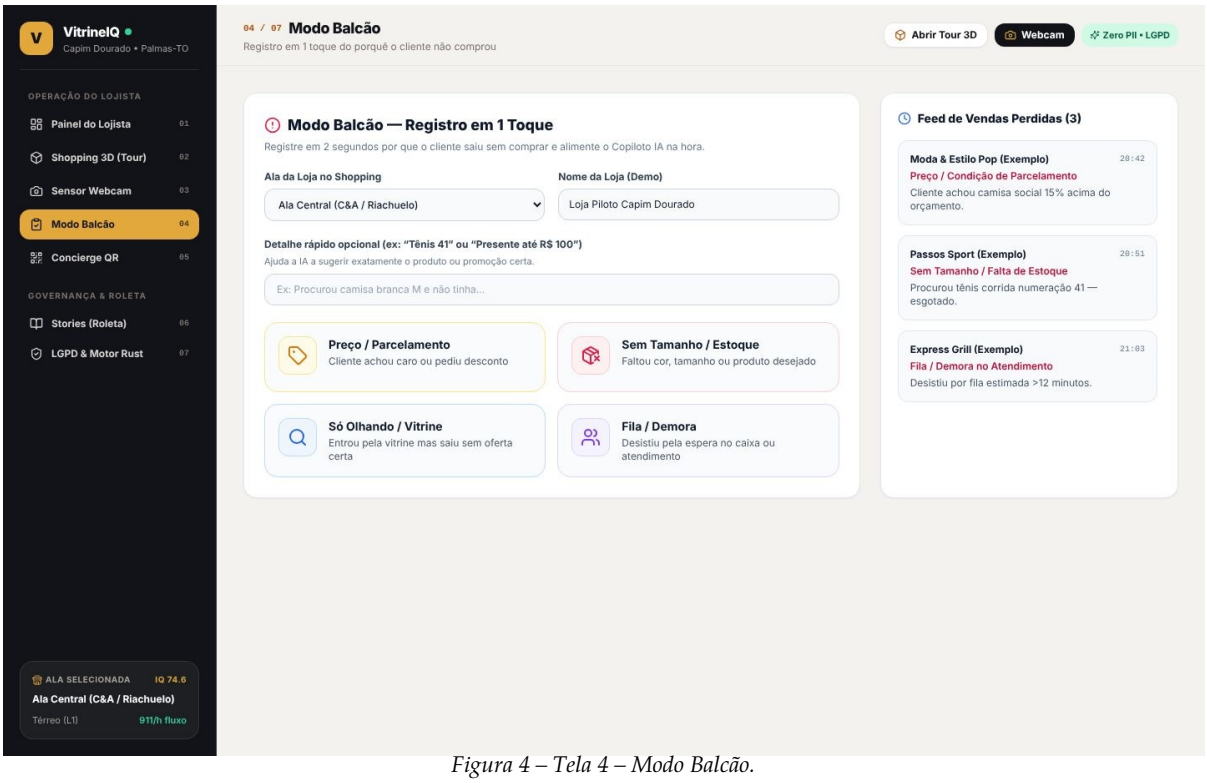


Figura 4 – Tela 4 – Modo Balcão.

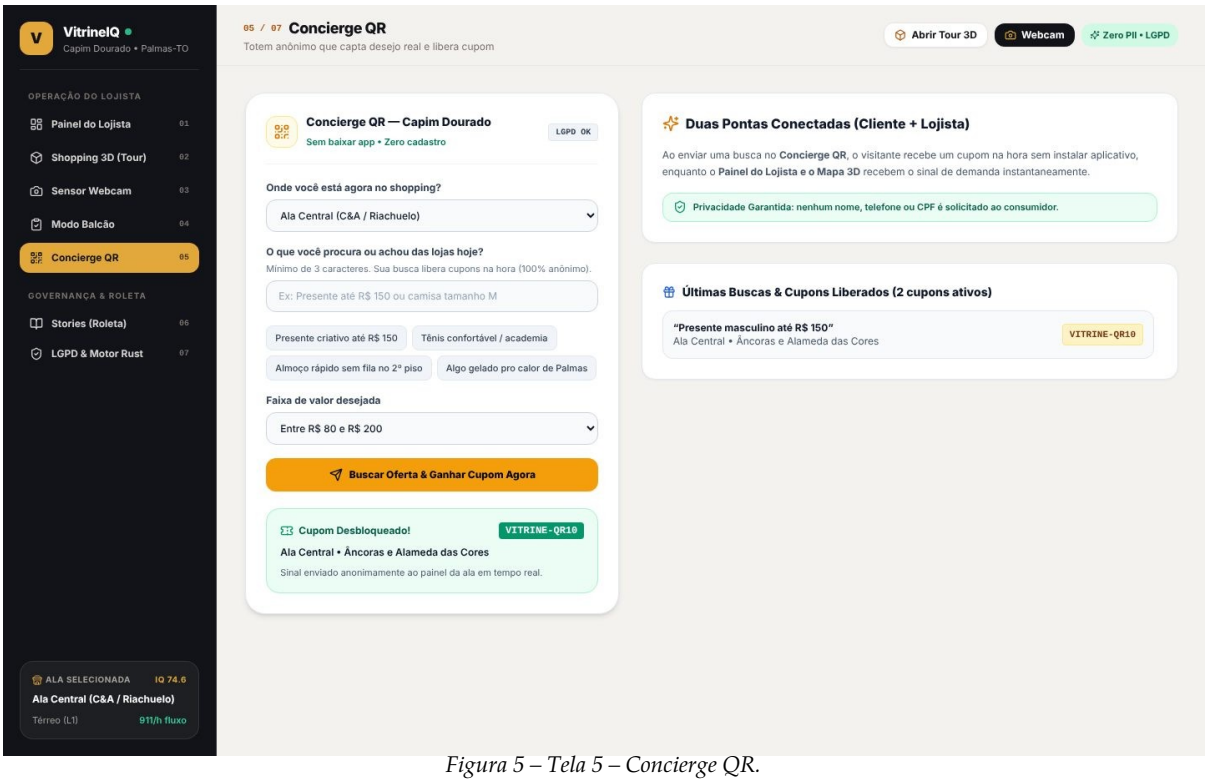


Figura 5 – Tela 5 – Concierge QR.

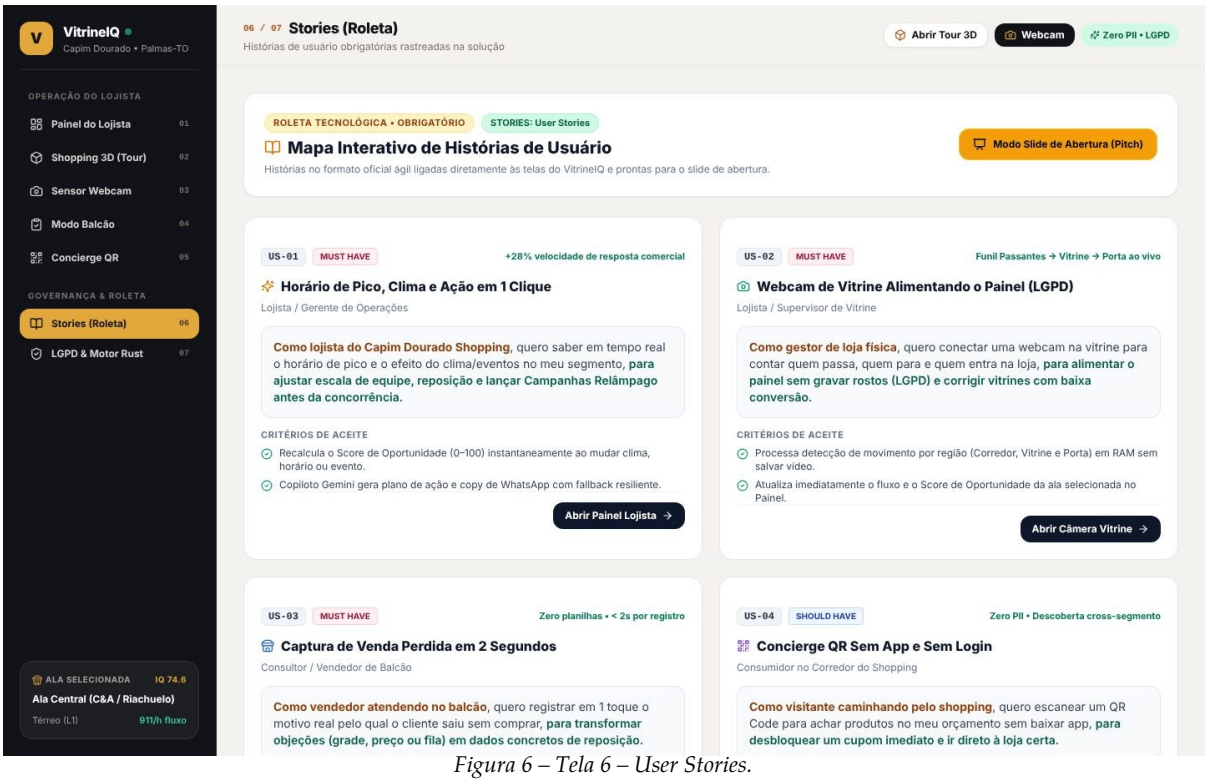


Figura 6 – Tela 6 – User Stories.

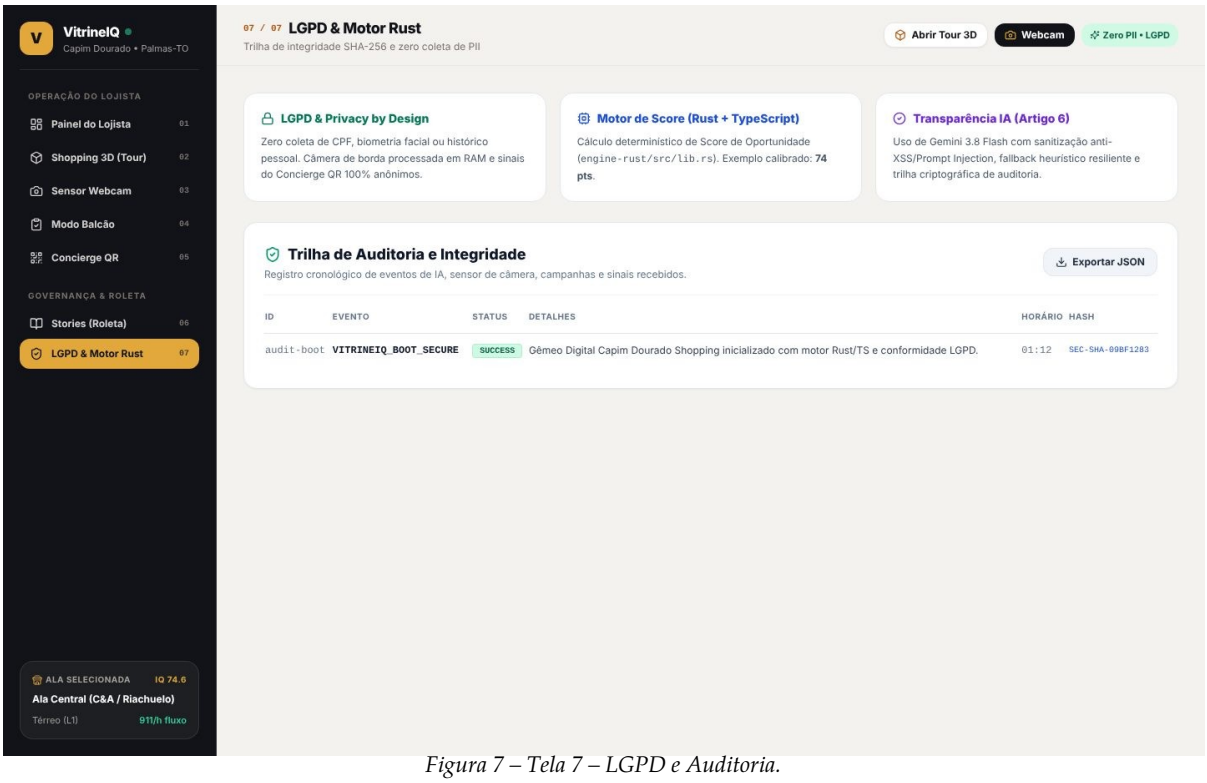


Figura 7 – Tela 7 – LGPD e Auditoria.

2. Documentação inicial do projeto

2.1 Problema, público e proposta de valor

Problema. Os lojistas do Capim Dourado Shopping, de diferentes segmentos, só enxergam as vendas da própria loja. Não sabem quem circula pelo shopping, por que o cliente entra e sai sem comprar, nem como horário, clima e eventos afetam seu desempenho.

Público-alvo. Lojistas e gerentes de lojas satélite do shopping, seus vendedores e a administração do shopping. A demonstração usa a persona de uma lojista de moda; a solução vale para qualquer segmento.

Proposta de valor. Transformar sinais anônimos do corredor, da vitrine e do balcão em decisões imediatas de vitrine, estoque, equipe e ofertas, dentro da LGPD.

2.2 Artefatos produzidos

Artefato	Onde está
Documento de Requisitos (RF, casos de uso, RNF, regras de negócio, glossário e User Stories)	Google Docs – Documento de Requisitos
Defesa do problema, Lean Canvas e backlog MoSCoW	Repositório, pasta missao-1
Código-fonte, testes e instruções de execução	Repositório no GitHub
Quadro Kanban (GitHub Projects)	Kanban do time
Protótipo publicado	vitrineiq-capim-dourado.vercel.app
Acordo de trabalho do time (OKRs, Kanban e daily)	Folha assinada pelos quatro integrantes e arquivo ACORDO_DE_TRABALHO_E_KANBAN.md

2.3 Fontes de dados

O protótipo usa dados simulados a partir da planta e do mix real do shopping, complementados por uma coleta real em pequena escala com lojistas durante o evento. A tabela mostra de onde cada dado vem no MVP e em produção.

Dado	No MVP (hackathon)	Em produção
Motivo de não compra	Modo Balcão, com teste junto a lojistas reais	Vendedor registra no tablet ou celular da loja
O que o visitante procura	Concierge QR (teste com voluntários)	QR fixo nas vitrines, com cupom como incentivo
Fluxo na vitrine	Webcam do notebook ou simulador	Webcam da loja, com aviso de monitoramento na vitrine
Clima e eventos	Valores simulados	API pública de clima e agenda do shopping e do cinema
Vendas por produto	Fora do MVP	Planilha exportada do sistema de caixa (PDV)
Percurso no shopping	Fora do MVP	Wi-Fi do shopping com portal cativo e consentimento

3. Linguagem de programação escolhida

O time usa duas linguagens, uma para cada lado da aplicação.

- **TypeScript com React (frontend):** TypeScript em modo strict com React 19, para a interface, o mapa 3D e o sensor de vitrine. A tipagem cobre os modelos de telemetria, as zonas (L1/L2), os eventos de balcão e as respostas da IA. O frontend já está construído e publicado.
- **Python (backend):** API em FastAPI que calcula o Score de Oportunidade (backend/app/engine.py), sanitiza as entradas e chama o Agente de IA guardando a chave da API no servidor. Escolhido por ser forte em dados e integração com IA, e por facilitar as evoluções previstas, como a importação de vendas em planilha (pandas).

4. Arquitetura proposta

A arquitetura tem três camadas e não coleta dados pessoais: o frontend em React capta os sinais anônimos e exibe os indicadores; o backend em Python calcula o score e é o único que conversa com a IA; o provedor de IA externo recebe apenas números agregados. Componentes tracejados são evoluções previstas.

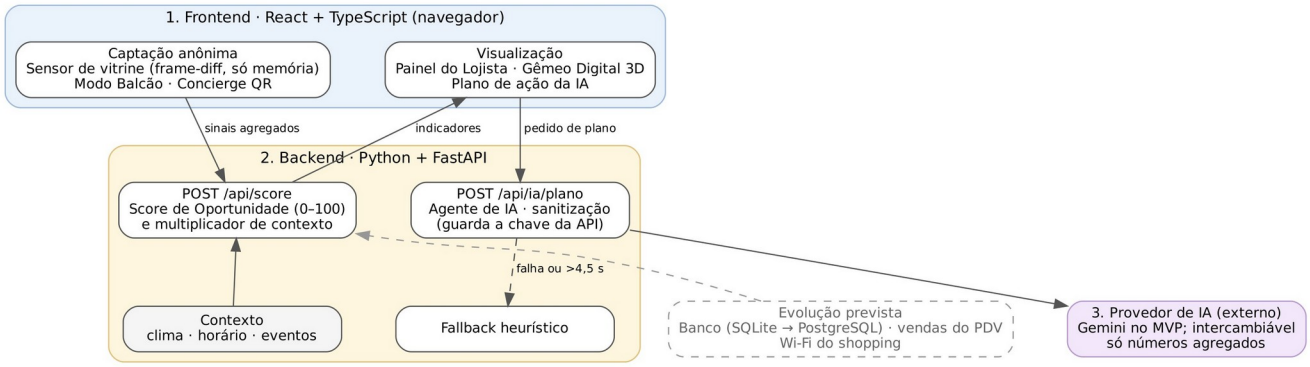


Figura 8 – Arquitetura do VitrineIQ.

- **1. Frontend (React + TypeScript, navegador):** o sensor óptico (WebRTC getUserMedia + frame-diff em Canvas) analisa a variação de pixels nas zonas corredor, vitrine e porta na memória do navegador; nenhum quadro é salvo ou enviado. O Concierge QR e o Modo Balcão captam os sinais ativos. O Painel do Lojista e o Gêmeo Digital 3D (Three.js) fazem a visualização.
- **2. Backend (Python + FastAPI):** POST /api/score calcula o Score de Oportunidade (0 a 100) e o multiplicador de contexto (clima de Palmas, horário e eventos do shopping). POST /api/ia/plano chama o Agente de IA. A documentação da API é gerada automaticamente em /docs e os testes usam pytest.
- **3. Provedor de IA (externo):** recebe só números agregados, nunca dados pessoais. A chave fica apenas no backend. O provedor é intercambiável: o MVP usa o Gemini, mas o backend pode trocar por outro modelo (OpenAI, Anthropic ou um modelo local) sem mudar o frontend. O endpoint bloqueia prompt injection, tem timeout de 4,5 s e, se a IA falhar, o fallback heurístico responde.

Evolução prevista: persistência dos registros em banco (SQLite → PostgreSQL), importação de vendas do PDV e percurso agregado pelo Wi-Fi do shopping.

5. Plataforma e tecnologias adotadas

Camada	Tecnologia	Papel na solução
Interface	React 19, TypeScript, Vite 6	Aplicação web responsiva para tablet do balcão, notebook da gerência e celular do visitante
Estilo	Tailwind CSS 3, Lucide Icons	Componentes visuais e identidade da interface
Mapa	Three.js (WebGL, OrbitControls)	Maquete 3D dos dois pavimentos com base na Planta Mix, com zoom, escada rolante e modo Street View
Sensor de vitrine	MediaDevices API, HTML5 Canvas	Detecção de movimento em vitrine e porta no navegador, sem gravação
API	Python, FastAPI, Uvicorn	Cálculo do score, sanitização e Agente de IA; documentação automática em /docs
Inteligência artificial	Agente de IA via API (Google Gemini no MVP; provedor intercambiável), fallback heurístico	Plano de ação, texto de cupom e sugestão de parceria
Hospedagem	Vercel	Publicação do frontend com HTTPS e deploy a cada commit
Qualidade	Vitest, pytest	Testes automatizados do frontend e do backend
Gestão	GitHub, GitHub Projects	Versionamento e quadro Kanban

6. Prática sorteada: User Stories

A prática sorteada para o time foi User Stories. Todos os requisitos são escritos como histórias no formato "Como <persona>, quero <ação>, para <benefício>", com critérios de aceitação no formato Dado / Quando / Então. A prática é aplicada do planejamento à apresentação:

1. **Planejamento:** as histórias formam o backlog priorizado em MoSCoW (Must, Should, Could) e definem o escopo do MVP.
2. **Organização:** cada história vira um cartão no Kanban, com responsável e critério de pronto.
3. **Desenvolvimento:** os commits citam a história que implementam, por exemplo "feat(US03): registro de venda perdida".
4. **Validação:** os critérios de aceitação são o checklist de testes; uma história só está pronta quando todos passam.
5. **No produto:** a tela User Stories mostra cada história e abre a funcionalidade que a implementa.

ID	História de usuário	Prioridade
US01	Como lojista, quero saber o horário de pico e o efeito do clima e dos eventos no meu segmento e gerar campanhas relâmpago, para ajustar equipe, estoque e promoções.	Must
US02	Como gerente de loja, quero uma webcam na vitrine que conte quem passa, para e entra, para alimentar o painel sem gravar imagens.	Must
US03	Como vendedor, quero registrar em segundos por que o cliente não comprou, para que faltas e objeções virem ação.	Must
US04	Como consumidor, quero dizer o que procuro pelo QR e receber um cupom sem cadastro, para encontrar o produto certo rapidamente.	Should
US05	Como gestor do shopping, quero ver os pavimentos em 3D com calor de demanda e espaços vagos, para identificar rotas e oportunidades.	Should

Os critérios de aceitação completos estão na seção 8 do Documento de Requisitos.

7. Evolução prevista (roadmap)

A solução foi pensada em três níveis. O primeiro funciona em qualquer loja sem depender do shopping; os seguintes ampliam os dados quando houver parceria.

Nível	Fontes de dados	O que passa a mostrar
1. Loja (MVP)	Modo Balcão, Concierge QR, sensor de vitrine, contexto	Funil da porta, motivos de não compra, horário de pico, plano de ação
1b. Persistência	Banco de dados (SQLite → PostgreSQL)	Histórico por loja e registros compartilhados entre aparelhos
2. Shopping	Wi-Fi com portal cativo e consentimento, contadores nas entradas	Percurso agregado, tempo por ala, visitas conjuntas entre lojas
3. Escala	Vendas do PDV, nota fiscal no app do shopping	Produtos mais e menos vendidos, giro de estoque, compras entre lojas

8. Equipe e uso de IA

Integrante	Responsabilidade
Arthur	Inteligência artificial e liderança técnica
Jessé	Frontend e telas
Leonardo	Coordenação, backend e documentação
Matheus	Negócio, validação com lojistas e pitch

Conforme o regulamento, o uso de IA generativa é declarado: o time usou assistentes de IA para apoio na ideação, pesquisa, estruturação da documentação e geração de código e testes. Todo o conteúdo foi revisado pelo time, que responde por ele. A declaração completa, com ferramentas e finalidades, acompanha a entrega final.